Techincal University of Denmark

02613 Python and High-Performance Computing

Wall Heating

GitHub Repository:

`https://github.com/IdaUgilt/PyHPC_Project`

Ida Wennergaard
s213309

Laura Bjerg Frandsen
s224406

Rasmus Niels Henriksen
s224399

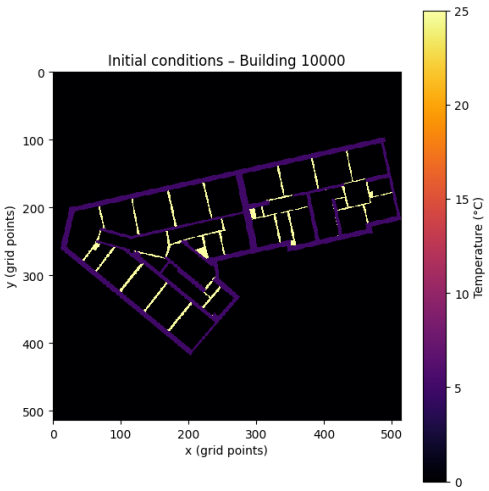
April 29, 2026

Note

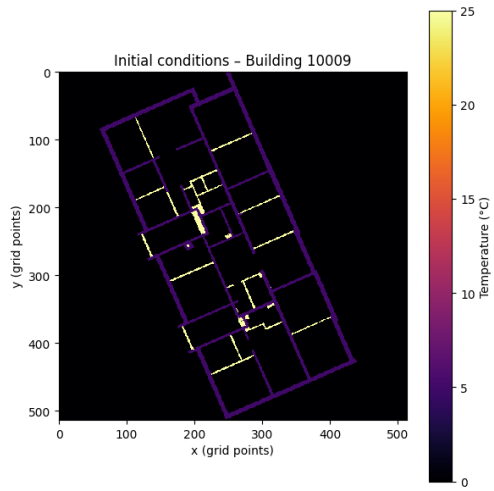
To ensure consistency, we have used the processor "XeonGold6226R" for all of our CPU computations on the HPC. For the GPU computations, we have put our job scripts in the queue `c02613`. We have used the environment `02613_2016` for the entire project.

1 Familiarize yourself with the data

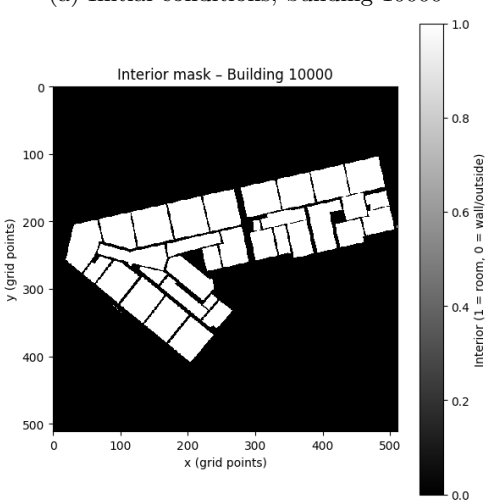
Figure 1 shows the input information we use for the Jacobi method, for two example buildings. The initial condition is u_0 and the second is interior mask. Only the points where the mask equals 1 are updated when performing the Jacobi method.



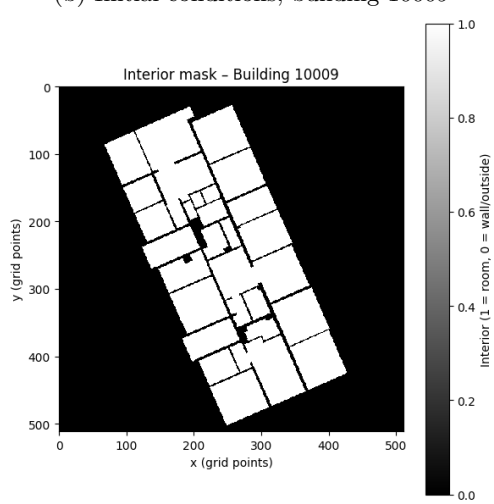
(a) Initial conditions, building 10000



(b) Initial conditions, building 10009



(c) Interior mask, building 10000



(d) Interior mask, building 10009

Figure 1: Input data for two example buildings. Top row: initial condition grids u_0. Bottom row: binary interior masks.

2 Estimated running time

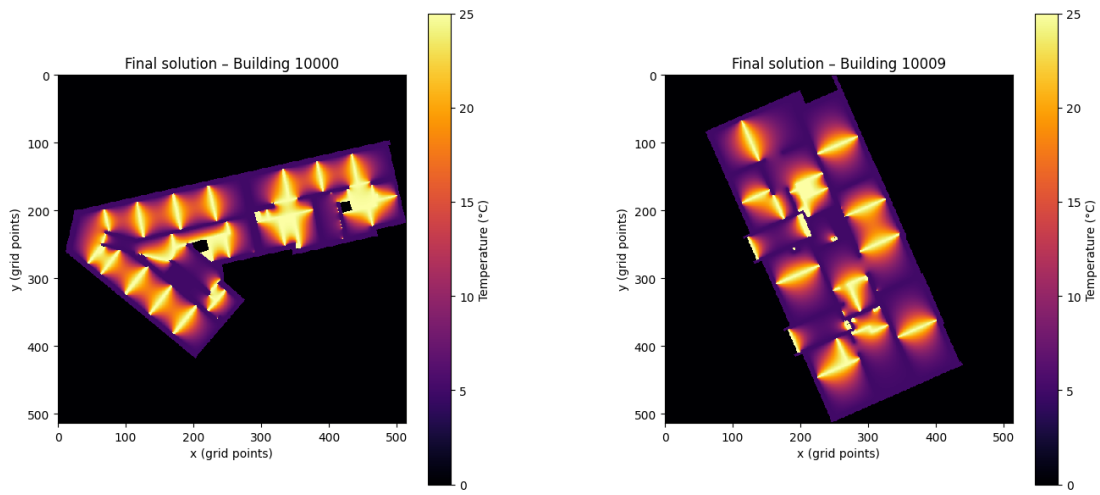
To estimate the total running time of the reference implementation, we run one time with the first 20 buildings. We use the function `perf_counter()`, to time the entire run of `__main__`.

The time it took to run 20 buildings was **230.9921 seconds**. The total running time for all floors can be estimated as

$$\frac{230.9921}{20} \cdot 4571 \approx \textbf{52,793.5 seconds} \approx \textbf{14.67 hours}.$$

3 Visualize the simulation results

Using the same building ids as in Section 1, we visualize the simulation results in Figure 2.



(a) Building 10000 (b) Building 10009

Figure 2: Final steady-state temperature distribution.

4 Profiling the Jacobi function

We use function profiling to identify problematic parts of the code. As seen in Figure 3, the third line in the table shows that the Jacobi function takes the longest time.

```
         110522 function calls (107632 primitive calls) in 8.005 seconds

   Ordered by: cumulative time

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
    117/1    0.001    0.000    8.005    8.005 {built-in method builtins.exec}
        1    0.004    0.004    8.005    8.005 simulate.py:1(<module>)
        1    7.615    7.615    7.773    7.773 simulate.py:15(jacobi)
       15    0.001    0.000    0.499    0.033 __init__.py:1(<module>)
```

Figure 3: Function profiling output of the provided script.

The Jacobi function iterates over each floor plan, updating interior temperatures until convergence. We use the kernprof command to line profile the Jacobi function. The output of the profiling is shown in Figure 4, as we have processed 5 buildings.

We see that line #20 is the most time consuming, as it uses 55.7% of the total run time. This line is the Jacobi update, in which we take the average of the 4 neighbors

Line #22 is the second most time consuming, which is the calculation of delta, which checks how much the solution changed this iteration. This takes 21.3% of the time. Line #23 and #21 are the third and fourth most time consuming. Respectively, it is the write-back of the new interior values into `u` using 12.0% of the time, and the mask gather of `u_new` into `u_new_interior` using 10.9% of the time. Together, the three masked operations on `interior_mask` take up about 44% of the runtime. The remaining lines (the copy, the loop, the convergence check), take under 1% of the time combined.

```
Total time: 49.3231 s
File: simulate.py
Function: jacobi at line 14

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
    14                                           @profile
    15                                           def jacobi(u, interior_mask, max_iter, atol=1e-6):
    16         5       2671.9    534.4      0.0      u = np.copy(u)
    17
    18     28423      14063.8      0.5      0.0      for i in range(max_iter):
    19                                                   # Compute average of left, right, up and down neighbors, see eq. (1)
    20     28423   27492545.1    967.3     55.7          u_new = 0.25 * (u[1:-1, :-2] + u[1:-1, 2:] + u[:-2, 1:-1] + u[2:, 1:-1])
    21     28423    5391060.3    189.7     10.9          u_new_interior = u_new[interior_mask]
    22     28423   10491981.5    369.1     21.3          delta = np.abs(u[1:-1, 1:-1][interior_mask] - u_new_interior).max()
    23     28423    5898496.3    207.5     12.0          u[1:-1, 1:-1][interior_mask] = u_new_interior
    24
    25     28423      32265.8      1.1      0.1          if delta < atol:
    26         5          3.2      0.6      0.0              break
    27         5          1.4      0.3      0.0      return u

 49.32 seconds - simulate.py:14 - jacobi
```

Figure 4: Line profiling of the Jacobi function.

5 Static parallelization

Working with 100 floor plans, we have parallelized the computations of the Jacobi function. The implementation is inspired by the chunked parallelization from exercise 2, week 5. The idea is to split the input data into predefined chunks of the same size. We define the same amount of chunks as we have processors.

To split the information into chunks, we use the NumPy function `np.array_split(array, amount_of_chunks)`. The function ensures that the floor plans are split as evenly as possible between the chunks. If the number of floor plans does not divide evenly, then the remainder is distributed one per chunk. If we e.g. have 3 processors to our 100 floor plans, then we will get two chunks with 33 floor plans and one chunk with 34 floor plans.

As the given `jacobi` function does not take a chunk as input, we define `jacobi_multiple`. This function takes a chunk of information as input, and using a for-loop, it passes it to the already defined `jacobi` function. The `jacobi_multiple` function is defined as

```
def jacobi_multiple(all_u0_chunk, all_interior_mask_chunk, max_iter, atol=1e-6):
    partial_all_u = np.empty((len(all_u0_chunk), 514, 514))
    for i, (u0, interior_mask) in enumerate(zip(all_u0_chunk, ...
        all_interior_mask_chunk)):
        u = jacobi(u0, interior_mask, max_iter, atol)
        partial_all_u[i] = u
    return partial_all_u
```

We can now perform the static parallelization. As seen in the code below, we set the amount of worker processes to the amount of available processors, and then we assign all tasks and their chunk of information to the worker processes. Lastly, we stack the results, forcing the workers to actually perform the tasks, and then we close and join the pool.

```
    pool = multiprocessing.Pool(n_proc)
    results_async = [pool.apply_async(jacobi_multiple, (all_u0_chunk[i],
                                        all_interior_mask_chunk[i],
                                        MAX_ITER, ABS_TOL))
                                        for i in range(n_proc)]

    all_u = np.vstack([r.get() for r in results_async])
    pool.close()
    pool.join()
```

5.a Speed-up achieved

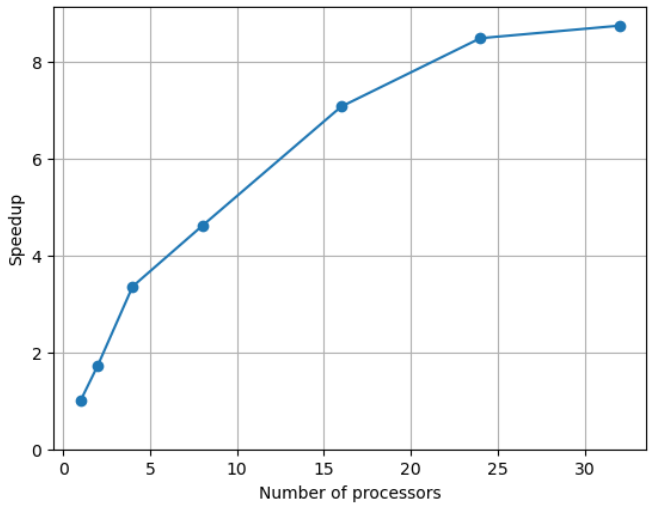


Figure 5: Computed speedup for static parallelization.

Figure 5 shows the speed-up as more workers are added with 1, 2, 4, 8, 16, 24 and 32 processors tested. We see that the increase in speed-up stabilizes at around 24 processors.

5.b Estimated parallel fraction

Amdahl's law is given as

$$S(p) = \frac{1}{(1-F) + \frac{1}{p}}.$$

We use the non-linear fitting algorithm Levenberg-Marquardt to fit our data to Amdahl's law, as shown on Figure 6. This gives the estimated parallel fraction $F = 0.916$.

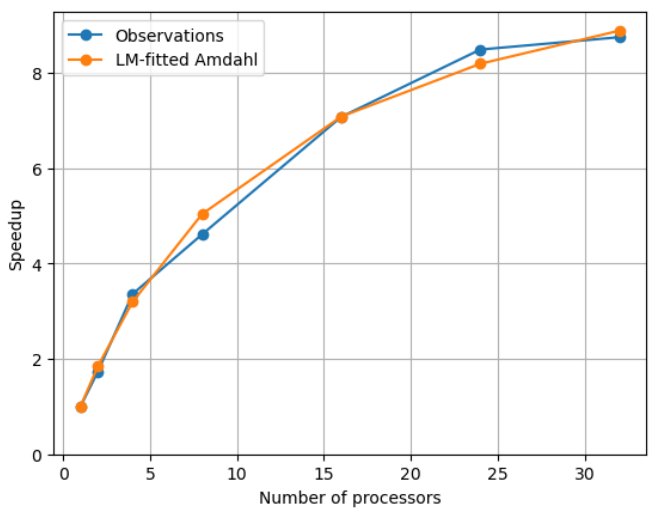


Figure 6: Computed speedup for static parallelization.

5.c Theoretical maximum speed-up

According to Amdahl's law, the theoretical maximum speedup as $p \to \infty$ is given by

$$S_{max} = \frac{1}{1-F} = \frac{1}{1-0.916} = 11.9.$$

The theoretical maximum speedup is thus 11.9. The maximum speedup we achieved was 8.75 which is 73% of the theoretical maximum. This was achieved using 32 cores.

5.d Time to process all 4571 buildings

There are 45.71 times more buildings when using the full dataset, compared to our current 100 buildings. Both the parallel and the non-parallel fraction would therefore also take 45.71 times longer.

Computing for the 100 buildings, the static parallelization using 32 cores takes 141 seconds, equivalent to 0.039 hours. As both the parallel and non-parallel fraction is extended by the same factor, we estimate that computations for the entire dataset will take 6445.7 seconds, equivalent to 1.8 hours.

6 Dynamic parallelization

Our dynamic parallelization is inspired by the full parallelization from exercise 2, week 5. In this implementation we do not use a helper function or chunk the data, as we did previously with static chunking. Instead, we create the same amount of worker processes and then create N tasks (where N is the 100 buildings we compute for). Then, we let the workers pick a new task from the queue when they are done with their task. This is done by using the following code.

```
    pool = multiprocessing.Pool(n_proc)
    results_async = [pool.apply_async(jacobi, (all_u0[i],
                                        all_interior_mask[i],
                                        MAX_ITER, ABS_TOL))
                                        for i in range(N)]

    all_u = np.stack([r.get() for r in results_async], axis=0)

    pool.close()
    pool.join()
```

6.a Is the dynamic faster than the static?

Figure 7 shows the difference in processing time for 100 building for the static and the dynamic parallelization. We see that the dynamic parallelization is faster than the static, but the difference between them decreases as the number of processors increases.

The dynamic parallelization is at most 81.2 seconds faster which is achieved using 2 cores. It is on average 26.8 seconds faster than the static. We expected the dynamic to be faster because it adapts to uneven workloads. In this project, some buildings might be quick to compute and some might not. Using dynamic parallelization therefore prevents the workers from sitting idle while others finish.

Figure 7 also shows a general trend of the two parallelization methods having a smaller difference as we use more processors. This makes sense because as we increase the amount of processors,

the running time becomes dominated by the overhead and the amount of tasks/buildings for each worker becomes smaller.

The time difference has a dip at four processors. This is a bit odd and unexpected, but could be because the data is divided better in the static model when there are four processors and it thus achieves a better performance.

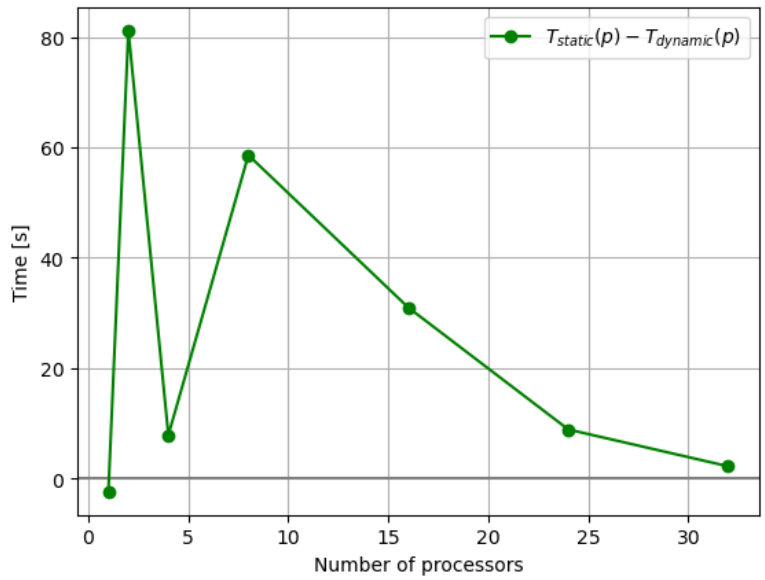


Figure 7: Difference in time to process the floor plans between the static and the dynamic parallelization for 100 buildings.

The total running time of the dynamic parallelization is estimated to 1.7 hours using 32 cores compared to 1.8 hours for the static parallelization. This is an improvement of 3.5 minutes. Hence, in practice the difference between the two implementations is rather small.

6.b Did the speed-up improve?

The speedup is computed and compared to the static parallelization in Figure 8. We see that the speedup is improved using dynamic parallelization compared to static parallelization. In the beginning the speedup increases quicker for the dynamic parallelization but the difference decreases as more processors are added.

The theoretical maximum speedup for the dynamic parallelization is $S_{max}^{\text{dyn}} = 13.6$ compared to $S_{max}^{\text{sta}} = 11.9$ for the static parallelization.

The dynamic parallelization is more efficient, but the difference decreases as more cores are added. This is most likely due to the dynamic parallelization having a larger overhead, as the worker processes must take each task, accumulating overhead time.

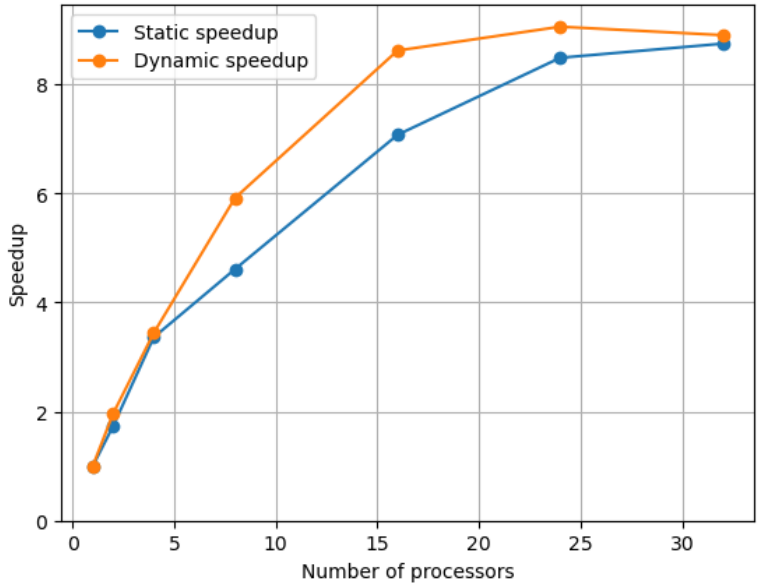


Figure 8: Speedup of the static and the dynamic parallelization.

7 Numba JIT

7.a Performance compared to the reference?

Hopefully, running this on the same small subset as the reference returns a much faster code.

To estimate the total running time of the reference implementation, we use the same method as in Section 2. Now, because we are working with Numba, we first kickstart the function with 1 building.

The time it took to kickstart the command with 1 building was **1.4437 seconds**. The time it took to run 20 buildings was **34.0395 seconds**. In total **35.4832 seconds** for 20 buildings. This is a speed up of 6.5, compared to the reference solution.

7.b Explain your function

Trying to rewrite the Jacobi function using Numba is very clever. We are running very simple operations for a large array for many iterations. Machine code is faster than Python for these simple tasks. The downside is that we need time to compile the first time over, but as we are running the function many more times, this turns out to be irrelevant.

To rewrite the Jacobi function, we need to remove the numpy-accessing way and rewrite it to a ”slower” for-loop. Machine code is very much cache-based, and accessing the original way is not cache-efficient. Rewriting with explicit loops removes temporary array allocations, allowing Numba to fuse the computations into a single cache-friendly code, which is what makes the machine code significantly faster. Explicitly writing out the loop, will in the end allow the machine code to be much more efficient.

The rewritten Jacobi function is

```
import numpy as np
from numba import jit


@jit(nopython=True)
def jacobi(u, interior_mask, max_iter, atol=1e-6):
    u = np.copy(u)
    row,col = np.shape(u)
    for _ in range(max_iter):
        ∆_max = 0.0
        for i in range(1,row-1):
            for j in range(1,col-1):
                if interior_mask[i-1,j-1]:
                    u_new = 0.25 * (u[i-1, j] + u[i+1, j] + u[i, j+1] + u[i, j-1])
                    ∆ = np.abs(u[i,j]-u_new)
                    u[i,j]= u_new
                    if ∆ > ∆_max:
                        ∆_max= ∆
        if ∆_max < atol:
            break
    return u
```

7.c Time to process all 4571 buildings

Assuming the time to process each building is the same for all buildings, the total time for all 4571 buildings would be

$$\frac{35.4832}{20} \cdot 4571 \approx \textbf{8107.6 seconds} \approx \textbf{2.25 hours}.$$

8 Custom CUDA Kernel with Numba

8.a Description of new solution

We implemented a custom CUDA kernel using Numba's cuda.jit decorator to perform the Jacobi iteration directly on the GPU. The solution consists of two functions: a helper function and a kernel function.

The helper function manages data transfer between the CPU and the GPU and repeatedly invokes the kernel. The kernel then performs a single iteration of the Jacobi solver, meaning that it computes one Jacobi update across all cells. The kernel is structured such that each thread maps to a single interior cell of the grid using a 2D thread grid. Each thread checks whether the assigned cell belongs to the interior using the interior mask array. If it does, then it computes the average of the four neighbors from the current state array `u`, and then writes the result to a separate output array `out`.

```
@jit(nopython=True)
@cuda.jit
def jacobi_kernel(u, mask, out):
    i, j = cuda.grid(2)

    if mask[i,j]:
        out[i+1, j+1] = 0.25 * (u[i, j+1] + u[i+2, j+1] + u[i+1, j+2] + u[i+1, j])
```

After each kernel call, the two device arrays are swapped so the output of one iteration becomes the input of the next.

The helper function allocates pinned host memory (`cuda.pinned_array`) for efficient data transfers and hereafter copies the arrays to the GPU (`cuda.to_device`). It runs the kernel in a loop for a fixed number of iterations (20,000). Thread blocks are configured as 16×16 threads, giving a grid of 32×32 blocks for the 512×512 interior domain.

```
def jacobi(u, interior_mask, max_iter, threads_per_grid=16):
    N = 512
    tpb = (threads_per_grid, threads_per_grid)
    bpg = (N // threads_per_grid, N // threads_per_grid)

    up = cuda.pinned_array((N+2,N+2), np.float64)
    maskp = cuda.pinned_array((N,N), np.bool_)
    outp = cuda.pinned_array((N+2,N+2), np.float64)

    up[:, :] = np.copy(u)
    maskp[:, :] = interior_mask
    outp[:,:] = np.copy(u)

    # Move data to gpu
    d_u = cuda.to_device(up)
    d_mask = cuda.to_device(maskp)
    d_out = cuda.to_device(outp)

    for i in range(max_iter):
        jacobi_kernel[bpg, tpb](d_u, d_mask, d_out)
        cuda.synchronize()
        d_u, d_out = d_out, d_u

    return d_u.copy_to_host()
```

8.b Performance compared to the reference

We timed this implementation and the reference implementation for the same 20 buildings. As for the reference implementation, we timed the entire run of the main function using `time.perfcounter()`. The GPU kernel completed in 25.59 seconds compared to 230.99 seconds for the CPU reference implementation. Meaning that this solution offers speed up of approximately 9 times.

8.c Time to process all 4571 buildings

There are 4571 floor plans in total. Assuming a linear scaling, the estimated processing time for all floor plans is

$$\frac{25.5889}{20} \cdot 4571 \approx \mathbf{5848.3\ seconds} \approx \mathbf{1.625\ hours}.$$

9 Adapt the reference solution to run on the GPU using CuPy

To adapt the reference solution, we simply import CuPy and change wherever numpy was used to CuPy.

9.a Performance compared to the reference

To estimate the total running time of the reference implementation, we use the same method as in Section 2. The time it took to run the code for 20 buildings was **57.4291 seconds**. This is 4 times faster than the original reference solution.

9.b Time to process all 4571 buildings

So, assuming equal time per building, the total time for all 4571 buildings would be:

$$\frac{57.4291}{20} \cdot 4571 \approx \textbf{13125.4 seconds} \approx \textbf{3.65 hours}.$$

9.c Surprising elements about the performance

Surprisingly, the CuPy solution is slower than both the parallelization and Numba implementation. This means the GPU is not optimized and we need to rewrite the code. For this, we would profile the function to understand which elements to optimize.

10 Profile the CuPy solution using the nsys profiler.

To run the profiler we write

```
nsys profile -o double_prof python simulate_cupy.py 20
nsys stats double_prof.nsys-rep
```

As the rest of the assignment, our focus is on the Jacobi function. Running on the original function with 20 buildings, we look at the GPU Kernel Summary.

Kernel	Time (%)	Instances	Grid Size
cupy_scan_naive	28.1	338	$512 \times 1 \times 1$
cupy_getitem_mask	21.0	370	$2048 \times 1 \times 1$
cupy_bsum_shfl	11.4	258	$512 \times 1 \times 1$
cupy_add_float64	9.5	143	$2048 \times 1 \times 1$

Table 1: GPU Kernel Summary for original code (20 buildings)

The GPU Kernel Summary is shown in Table 1. The thread/blocks for GPU memory map stuff onto Streaming Multiprocessors, which is the problem we should adress. The data must be large enough to keep all the multiprocessors busy. The profiler shows kernels such as `cupy_getitem_mask` being launched 370 times with a grid size of only $2048 \times 1 \times 1$. By batching all buildings into a single 3D array, the grid expands and the kernel calls are reduced.

Table 2 shows the GPU Kernel Summary with the optimized code. The number of times we call the kernel is significantly lower and the grid size is a lot bigger.

Kernel	Time (%)	Instances	Grid Size
cupy_add_float64	25.3	19	$40960 \times 1 \times 1$
cupy_scan_naive	22.8	26	$10240 \times 1 \times 1$
cupy_getitem_mask	19.6	29	$40960 \times 1 \times 1$
cupy_subtract_float64	8.8	21	$12540 \times 1 \times 1$

Table 2: GPU Kernel Summary for code with batched Jacobi(20 buildings).

The time it took to run 20 buildings for this revised code was **24.2761 seconds**. This is an improved, compared to the previous 57.4291 seconds. For all it would be

$$\frac{24.2761}{20} \cdot 4571 \approx \textbf{5548.2 seconds} \approx \textbf{1.54 hours}.$$

12 Process all floor plans

We have chosen to process all floor plans using our dynamic parallelization. We have chosen this as there are a more limited amount of GPUs in the HPC, and we do not want to fill up the queue unnecessarily when there are only a small difference between the GPU batched CuPy and the CPU dynamic parallelization. We process using 16 cores, as the speedup achieved using 24 and 32 cores were insignificant, as we saw on Figure 8.

We estimated earlier that the parallelization would use 1.7 hours to process all buildings using 32 cores. When processing for all buildings using 16 cores, the run time from the job script is 2.2 hours. This increase in time is most likely due to two things. First, we observed the run time of the first estimation using Python's `time.perfcounter()` which measures the CPU time. When processing all buildings, we did not measure the time and instead used the run time provided by the output file from the job script. Second, the run time is a bit slower using 16 cores, compared to the 32 cores used for the estimate.

12.a Distribution of the mean temperatures

The histogram in Figure 9 shows the distribution of the mean temperatures. It indicates a negatively skewed normal distribution of the mean temperatures. This is to be expected, as the Central Limit Theorem states that the distribution of the sample means will approach a normal distribution, when the amount of samples are increased.

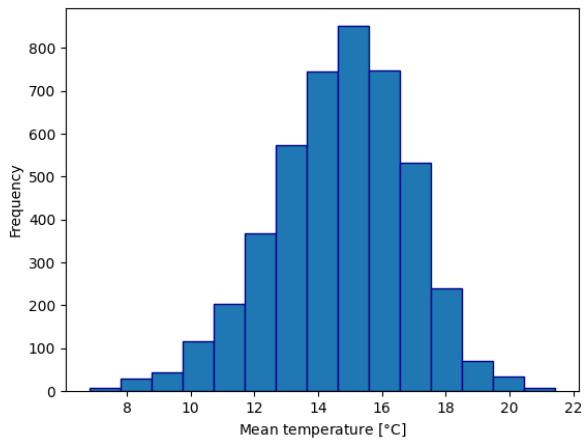


Figure 9: Histogram showing the distribution of the mean temperatures.

12.b Average mean temperature

The average of the mean temperatures of the buildings is 14.69 °C.

12.c Average temperature standard deviation

The average temperature standard deviation is 6.80 °C.

12.d Buildings with at least 50% of their area above 18ºC

804 buildings had at least 50% of their area above 18 °C.

12.e Buildings with at least 50% of their area below 15ºC

2471 buildings had at least 50% of their area below 15 °C.